

Backlog técnico do MeasureSoftGram

Levantamento de gargalos, defeitos ativos e dívida técnica nos sete repositórios do projeto. Cada item está redigido para virar issue: tem escopo fechado, caminho do arquivo e critério de aceite.

6 CRÍTICOS	14 ALTOS	14 MÉDIOS	7 BAIXOS	41 ITENS NO TOTAL
3 QUEBRADOS AGORA				

E1 Quebrado agora

Três itens que já falham hoje. Antes de qualquer feature nova, isto precisa estar verde — todo o resto do backlog é construído em cima de um CI confiável.

MSG-01 Fixture desatualizada derruba dois testes no `develop` do Parser

CRÍTICO

Parser

O commit `f9bc01c` (24/08/2026) removeu o prefixo de semestre dos mocks, trocando `2023-1-MeasureSoftGram-DOC` por `MeasureSoftGram-DOC` no arquivo de teste — **mas não atualizou a fixture**. As linhas 16 e 34 de `expected_return_values.py` ainda carregam o nome antigo, e `test_extract_method_all_filters` e `test_extract_method_date_none` falham.

Foi integrado ao `develop` pelo PR #24 no mesmo dia. O sintoma de fundo é mais grave que o defeito: **o resultado do CI não está sendo verificado antes do merge**.

CRITÉRIO DE ACEITE

`pytest` passa integralmente no `develop`, e o branch fica protegido exigindo CI verde para merge.

Esforço **15 min** `tests/mockfiles/expected_return_values.py:16,34`

MSG-02 Restrição frouxa de dependência quebra o CLI em instalação limpa

CRÍTICO

CLI

O CLI declara `msggram-core~=1.5.3`, que resolve para **1.5.4**. Nessa versão, três testes falham — `test_calculate_github`, `test_calculate_csv_output` e `test_calculate_json_output` — retornando `0.9677...` onde se espera `0.0322...`.

Verificado por execução: fixando `msggram-core==1.5.3`, os **127 testes unitários passam**. Qualquer instalação limpa hoje, inclusive a de CI, está vermelha.

CRITÉRIO DE ACEITE

Versão do Core fixada com `==` (não `~=`) e suíte verde em ambiente limpo. Depende da decisão tomada em MSG-05.

Esforço **1 h** `pyproject.toml · requirements.txt`

MSG-03 GET /users/ expõe e-mail de todos os usuários sem autenticação

CRÍTICO

Service

A configuração `REST_FRAMEWORK` define `DEFAULT_AUTHENTICATION_CLASSES` mas **não define** `DEFAULT_PERMISSION_CLASSES`. O padrão do DRF nesse caso é `AllowAny`: todo viewset sem permissão explícita fica aberto.

O pior caso é `UserListViewSet`, roteado em `GET /users/`, que serializa `id`, `username`, `first_name`, `last_name` e **email de todos os cadastrados**. Também estão abertos `goals`, `release_configuration`, `entity_trees`, `math_model`, `measures`, `subcharacteristics` e `ReleaseListAllModelViewSet`.

CRITÉRIO DE ACEITE

`DEFAULT_PERMISSION_CLASSES = (IsAuthenticated,)` configurado; endpoints de badge marcados explicitamente com `AllowAny`; teste automatizado garantindo `401` em `GET /users/` sem token.

Esforço **2 h** `src/config/settings/base.py:205 · src/accounts/views.py:95`

E2 Integridade do modelo de qualidade

O problema mais grave do projeto. O motor de cálculo publicado não corresponde ao código versionado, e os dois modos de uso produzem notas diferentes para o mesmo software.

MSG-04 Código do `msggram-core 1.5.4` publicado no PyPI não está versionado

CRÍTICO

Core

O repositório declara a versão **1.5.3** e o código corresponde a ela. No PyPI existe **1.5.4**, publicado a partir de código ausente do repositório central. A comparação dos dois pacotes revela correções semânticas reais:

```
# aggregated_normalized_measures.py - medida ci_feedback_time
- gain_interpretation=1,
+ gain_interpretation=-1,      ← sentido invertido
- max_threshold: float = 0.33,
+ max_threshold: float = 0.66, ← limiar dobrado
+ if int(total_builds) == 0:
+     return NO_BUILDS_NEUTRAL_SCORE ← issue #24

# util/run_time_data_operations.py - bug de matriz não-quadrada
- return abs(sum / (size * size))
+ return abs(sum / (size_1 * size_2))
```

Enquanto isso não for resolvido, **a nota de qualidade que o projeto produz não é auditável** — não dá para reproduzi-la a partir do código versionado.

CRITÉRIO DE ACEITE

Código do 1.5.4 localizado, revisado e commitado no repositório Core, com *tag* correspondente. Publicação no PyPI passa a sair exclusivamente do repositório central via CI.

Esforço **1 dia** MeasureSoftGram-Core

MSG-05 Três versões do Core em circulação simultânea

CRÍTICO

Transversal

O Service fixa `msggram-core==1.4.9`; o CLI resolve para `1.5.4`; o repositório versiona `1.5.3`. A documentação de arquitetura afirma que os dois modos de uso “compartilham o mesmo motor de cálculo” — na prática, **o fluxo hospedado e o fluxo offline calculam notas diferentes para o mesmo repositório**.

CRITÉRIO DE ACEITE

Service e CLI apontando para a mesma versão do Core, com teste de contrato que compara a saída dos dois caminhos para um repositório de referência.

Esforço **3 dias** Service · CLI · Core

MSG-06 **Testes do CLI codificam a semântica antiga de** `ci_feedback_time` **ALTO** CLI

As três falhas de MSG-02 não são um defeito do 1.5.4 — são o CLI *não tendo acompanhado uma correção*. Os valores esperados nos testes refletem o sentido invertido da métrica, que o Core corrigiu. Fixar a versão resolve o sintoma; atualizar as expectativas resolve a causa.

CRITÉRIO DE ACEITE

Valores esperados recalculados sob a semântica correta, com comentário no teste explicando o sentido de `gain_interpretation`.

Esforço **3 h** `tests/unit/test_calculate.py`

MSG-07 **Sem teste de contrato entre Core, Service e CLI** **ALTO** Transversal

Nada no CI detecta que os consumidores do Core divergiram. MSG-04, MSG-05 e MSG-06 são todos sintomas da mesma lacuna: **não existe verificação de que os dois caminhos de cálculo concordam**.

CRITÉRIO DE ACEITE

Suíte que roda o mesmo conjunto de métricas pelo Service e pelo CLI e falha se as notas divergirem além da tolerância de ponto flutuante.

Esforço **1 semana** novo · integração

E3 Gargalos de performance

O cálculo do TSQMI é o caminho quente do sistema. Hoje ele faz uma consulta ao banco por item, em quatro níveis aninhados, sobre uma tabela sem índice.

MSG-08 N+1 em cascata de quatro níveis no cálculo do modelo

ALTO

Service

O cálculo percorre características → subcaracterísticas → medidas → métricas, e **em cada nível itera sobre `.all()` disparando uma consulta por item**. O código já reconhece o problema:

```
# src/measures/models.py:44
TODO: - Melhorar a query para o banco de dados.
      - Desconfio que aqui esteja rolando vários inner joins

for supported_metric in self.metrics.all():
    value = supported_metric.get_latest_metric_value(repository) ← 1 query cada
```

O mesmo padrão se repete em `subcharacteristics/models.py:64` e `characteristics/models.py:49,73`. As views de `organizations` já receberam `select_related` e `prefetch_related` no ciclo 2026.1, mas **a camada de modelos ficou de fora** — a otimização é parcial.

CRITÉRIO DE ACEITE

Cálculo completo de um repositório executa em número constante de consultas, verificado com `assertNumQueries` em teste de regressão.

Esforço 1 semana

`src/measures/models.py:39` · `src/subcharacteristics/models.py:50` · `src/characteristics/models.py:38`

MSG-09 CollectedMetric sem índice, com ordenação global no Meta

ALTO

Service

É a tabela de maior volume do sistema — uma linha por métrica, por arquivo, por coleta. O modelo declara `ordering = ["-created_at"]` no `Meta`, o que **impõe ordenação a todo queryset**, e as consultas filtram por `repository`, `metric` e faixa de `created_at`.

Nenhum desses campos tem `db_index=True`, e não há índice composto. Enquanto a base é pequena o custo não aparece; ele cresce com o histórico.

CRITÉRIO DE ACEITE

Índice composto sobre `(repository, metric, -created_at)` criado por migration, com `EXPLAIN` antes e depois registrado na issue.

Esforço 4 h `src/metrics/models.py:144-201`

MSG-10 `parser.py` grava JSON inválido em silêncio quando o Sonar falha

MÉDIO

Transversal

O script que coleta métricas do SonarCloud em todos os repositórios faz `requests.get` **sem timeout, sem verificar o status e sem tratar exceção**. Se a API responder 401, 429 ou 500, o corpo do erro é serializado como se fosse dado válido e anexado à release.

CRITÉRIO DE ACEITE

Timeout explícito, `raise_for_status()` e falha ruidosa do job quando a coleta não retorna métricas.

Esforço 2 h `parser.py` (6 repositórios)

E4 Segurança

Além de MSG-03, cinco itens no Service e no Front. Dois são risco presente; três são risco latente que vira incidente na primeira distração.

MSG-11 Django 4.0.6 sem suporte desde abril de 2023

CRÍTICO

Service

Três anos sem correções de segurança, rodando em produção sobre Python 3.12. A migração para a série **4.2 LTS** é o mínimo; 5.x é o alvo desejável.

CRITÉRIO DE ACEITE

Django em versão suportada, suíte verde e `manage.py check --deploy` sem avisos críticos.

Esforço 1 semana `pyproject.toml`

MSG-12 Token OAuth do GitHub armazenado em texto plano

ALTO

Service

`github_access_token` é um `CharField(max_length=255)` comum na tabela de usuários. É credencial de terceiro, sem cifragem — qualquer *dump* de banco ou acesso ao admin entrega os tokens de todos os usuários.

CRITÉRIO DE ACEITE

Campo cifrado em repouso e ocultado no Django Admin, com migration de dados para os registros existentes.

Esforço 1 dia `src/accounts/models.py`

MSG-13 Produção sem TLS: tokens de autenticação trafegam em claro**ALTO**

Service

O módulo `production.py` documenta a escolha com honestidade, mas entrega `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE` e `CSRF_COOKIE_SECURE` em `False`, e `SECURE_HSTS_SECONDS` em zero — porque o deploy fica atrás de proxy HTTP sem terminação TLS.

Combinado com `TokenAuthentication`, isso significa credencial em claro na rede. **As flags já estão prontas e parametrizadas por env** — falta apenas o certificado na borda.

CRITÉRIO DE ACEITE

TLS terminado na borda e as quatro flags ligadas em produção, com HSTS de um ano.

Esforço **1 dia** `src/config/settings/production.py` · `infra`

MSG-14 .gitignore do Front não cobre o diretório .envs/**MÉDIO**

Front

O `.gitignore` ignora `.env` e `.env.local` na raiz, mas **não o diretório `.envs/`** — onde os arquivos de ambiente do projeto realmente vivem. Hoje eles não contêm segredo, mas o primeiro que colocar uma chave ali vai versioná-la sem aviso.

CRITÉRIO DE ACEITE

`.envs/` ignorado, com `.envs/.env.example` versionado como referência.

Esforço **15 min** `.gitignore:29-35`

MSG-15 debug_toolbar em INSTALLED_APPS base e nas dependências de produção**MÉDIO**

Service

A ferramenta figura em `INSTALLED_APPS` e `MIDDLEWARE` do módulo `base` — não apenas no de desenvolvimento — e consta como dependência de produção no `pyproject.toml`. O `middleware` se desliga sozinho com `DEBUG=False`, mas é superfície desnecessária na imagem publicada.

CRITÉRIO DE ACEITE

`debug_toolbar` movido para `dev.py` e para o grupo de dependências de desenvolvimento.

Esforço **2 h** `src/config/settings/base.py:71,111`

MSG-16 CORS e CSRF com padrões apontando para infraestrutura desativada

BAIXO

Service

`CSRF_TRUSTED_ORIGINS` tem por padrão um domínio Railway do ciclo 2023-2 e `CORS_ALLOWED_ORIGINS` aponta para um Vercel de 2024-1. São domínios que o projeto não controla mais — se alguém os registrar, viram origem confiável.

CRITÉRIO DE ACEITE

Padrões trocados por lista vazia, obrigando configuração explícita por variável de ambiente.

Esforço 1 h `src/config/settings/base.py:34-48`

E5 Integração contínua

O Service foi modernizado em 2026.1 e virou o padrão de referência. Os outros seis repositórios ficaram em 2024, com ferramental descontinuado e nenhum portão bloqueante.

MSG-17 Nenhum portão de qualidade bloqueia merge

ALTO

Transversal

Os quatro arquivos `codecov.yml` — CLI, Core, Parser e Service — são idênticos e marcam `informational: true` em `project` e `patch`. O `.coveragerc` do Service não define `fail_under`. Na prática, **cobertura nunca reprovava um pull request**.

É o que permitiu MSG-01 entrar no `develop`. Num projeto cuja finalidade é medir qualidade de software, é a contradição mais visível do backlog.

CRITÉRIO DE ACEITE

Portão de `patch` bloqueante em todos os repositórios e proteção de branch exigindo CI verde no `develop`.

Esforço 4 h `codecov.yml` (4 repos) · configuração do GitHub

MSG-18 Action declara runtime `node16`, já removido pelo GitHub

ALTO

Action

O manifesto `action.yml` declara `using: 'node16'`. O GitHub migra automaticamente para node20 com aviso de descontinuação, mas o suporte tem prazo — **quando cair, todo repositório que usa a Action perde a coleta de métricas de uma vez**.

CRITÉRIO DE ACEITE

`using: 'node20'` no manifesto, `dist/` regenerado e execução validada em repositório de teste.

Esforço 3 h `action.yml:36`

MSG-19 Workflow do Core referencia repositório renomeado

ALTO

Core

`msggram-analysis.yml` usa `fga-eps-mds/2023-2-MeasureSoftGram-Action@v2.2` — repositório que foi renomeado. O workflow que faz o projeto se auto-analisar está quebrado.

CRITÉRIO DE ACEITE

Referência atualizada e execução bem-sucedida registrada na aba Actions.

Esforço **30 min** `.github/workflows/msggram-analysis.yml:14`

MSG-20 Actions descontinuadas em cinco repositórios

MÉDIO

Transversal

`actions/checkout@v3`, `actions/setup-node@v3` e `actions/setup-python@v4` aparecem em Front, Core, Action, Parser e CLI. Todas descontinuadas. O Service já usa v4/v5 **fixadas por SHA** — é o padrão a replicar.

CRITÉRIO DE ACEITE

Todas as actions em versão suportada e fixadas por SHA, como no Service.

Esforço **1 dia** `.github/workflows/` (5 repos)

MSG-21 Action do SonarCloud arquivada e fixada em `@master`

MÉDIO

Transversal

Front, Core e Action usam `SonarSource/sonarcloud-github-action@master`. A action foi **arquivada pela SonarSource** e substituída por `sonarqube-scan-action`. Fixar em `@master` é ainda risco de cadeia de suprimentos: qualquer commit no upstream entra no pipeline sem revisão.

CRITÉRIO DE ACEITE

Migração para `sonarqube-scan-action` fixada por SHA.

Esforço **4 h** `build.yml` (Front, Core, Action)

MSG-22 Chaves do Sonar divergem em três padrões diferentes

MÉDIO

Transversal

Service e Parser usam `MeasureSoftGram-*`; o CLI usa `2026.1-*`; Core, Front e Action seguem em `2024.1-*`. A organização também difere: o Service aponta para `fga-eps-mds`, todos os demais para `fga-eps-mds-1`. **O histórico de qualidade do projeto está fragmentado em três lugares.**

CRITÉRIO DE ACEITE

Todos os repositórios sob a mesma organização e convenção de chave, com histórico consolidado ou migração documentada.

Esforço **1 dia** `sonar-project.properties` (6 repos)

MSG-23 Nenhum repositório tem Dependabot ou Renovate

MÉDIO

Transversal

Não existe `.github/dependabot.yml` em nenhum dos sete repositórios. É a razão estrutural pela qual MSG-11, MSG-18, MSG-20 e MSG-21 puderam envelhecer até virarem trabalho de semanas — **sem automação, atualizar dependência depende de alguém lembrar.**

CRITÉRIO DE ACEITE

Dependabot configurado nos sete repositórios, com agrupamento de atualizações e alertas de segurança ligados.

Esforço **4 h** `.github/dependabot.yml` (novo)

MSG-24 Parser removeu a análise estática do CI

MÉDIO

Parser

O commit `fix: remove etapa do SonarCloud no CI`, de 24/08/2026, tirou o Sonar do pipeline — provavelmente para destravar o build durante a migração de chaves. É perda de portão que não foi reposta.

CRITÉRIO DE ACEITE

Análise estática restabelecida sob a chave correta, resolvido junto com MSG-22.

Esforço **2 h** `.github/workflows/test.yml`

MSG-25 Sem **CODEOWNERS** em nenhum repositório

BAIXO

Transversal

Não há atribuição automática de revisor. Com equipe que rotaciona a cada semestre, é o que faz o conhecimento de cada módulo se perder na virada.

CRITÉRIO DE ACEITE

CODEOWNERS por área em cada repositório, com revisão obrigatória na proteção de branch.

Esforço 2 h .github/CODEOWNERS (novo)

MSG-26 Action sem templates de issue e sem **CONTRIBUTING**

BAIXO

Action

CLI, Core, Parser e Service têm **ISSUE_TEMPLATE** e **pull_request_template.md**; o Front tem só o de PR; a Action não tem nenhum. A padronização de contribuição está incompleta.

CRITÉRIO DE ACEITE

Templates equalizados nos sete repositórios, referenciando o guia central de contribuição.

Esforço 2 h .github/ (Action, Front, Docs)

E6 Duplicação e reprodutibilidade

Código copiado entre repositórios que já divergiu, e builds que não produzem o mesmo resultado duas vezes.

MSG-27 **parser.py** copiado em seis repositórios, em quatro variantes divergentes

ALTO

Transversal

O mesmo script de coleta existe em Action, CLI, Core, Front, Parser e Service. Comparando as somas de verificação: **Parser e Service são idênticos; Core e Action divergem no conjunto de métricas coletadas; Front e CLI são outras duas variantes.**

A versão da Action coleta **test_success_density** e **reliability_rating**, que as outras não coletam. Isso significa que **repositórios diferentes do mesmo projeto são medidos com conjuntos de métricas diferentes** — comprometendo a comparabilidade entre eles.

CRITÉRIO DE ACEITE

Script único publicado como pacote ou workflow reutilizável, consumido pelos seis repositórios, com conjunto de métricas idêntico.

Esforço 3 dias parser.py (6 repos)

MSG-28 **Front e Docs versionam `yarn.lock` e `package-lock.json` ao mesmo tempo**

MÉDIO

Front

Dois lockfiles no mesmo repositório significam que **a árvore de dependências instalada depende de qual gerenciador o desenvolvedor usou**. Os dois arquivos divergem com o tempo e ninguém percebe até um bug só reproduzir na máquina de alguém.

CRITÉRIO DE ACEITE

Um único gerenciador escolhido por repositório, lockfile órfão removido e a escolha documentada no `CONTRIBUTING`.

Esforço **3 h** Front · Docs

MSG-29

Dockerfile do Front constrói para produção e roda o servidor de desenvolvimento

MÉDIO

Front

```
FROM node:lts-alpine      ← tag móvel: build não é reproduzível
RUN yarn build             ← constrói o bundle de produção...
CMD [ "yarn", "dev" ]      ← ...e roda o dev server, descartando-o
```

Além do desperdício, `node:lts-alpine` hoje resolve para uma versão muito além do que o Next 12 suporta, e o `docker-compose.yml` monta `./usr/src/` por cima da imagem, apagando o `node_modules` instalado no build. Sem multi-estágio e sem usuário não-root.

CRITÉRIO DE ACEITE

Imagem multi-estágio com tag de Node fixada, `CMD` coerente com o alvo e usuário não-root — espelhando o Dockerfile do Service.

Esforço **1 dia** Dockerfile · docker-compose.yml

MSG-30 **`dist/` versionado e defasado em quase dois anos**

MÉDIO

Action

O bundle compilado está no repositório e foi gerado pela última vez em **10/09/2024**. É o que efetivamente roda quando alguém usa a Action — ou seja, **o código-fonte em `src/` e o comportamento real podem já ter divergido**, sem nada que verifique.

CRITÉRIO DE ACEITE

Job de CI que regeira o `dist/` e falha se o resultado diferir do commitado.

Esforço **4 h** `dist/` · `.github/workflows/`

MSG-31 Dependências de teste declaradas como dependências de produção

BAIXO

Action

`axios-mock-adapter`, `mock-fs` e `jest-sonar` estão em `dependencies`, não em `devDependencies` — entram no pacote distribuído. O `package.json` também ainda se chama `2023-1-measuresoftgram-action`, com `repository`, `bugs` e `homepage` apontando para o repositório antigo.

CRITÉRIO DE ACEITE

Dependências reclassificadas e metadados do pacote corrigidos.

Esforço 1 h package.json

MSG-32 Arquivos gerados e de sistema versionados no Service

BAIXO

Service

`.DS_Store`, `output_sonar.json` e `test.json` estão rastreados pelo git, e o `.gitignore` não cobre `.DS_Store`. São artefatos gerados que geram conflito de merge sem nenhum benefício.

CRITÉRIO DE ACEITE

Arquivos removidos do rastreamento e `.gitignore` completado.

Esforço 30 min .gitignore · raiz do repositório

E7 Qualidade de código e testes

O Python do projeto está limpo — zero apontamentos de flake8 nos quatro repositórios. A dívida se concentra no Front, que é a maior superfície e a menos tocada.

MSG-33 Next.js 12.2.2 fora de suporte, com dependências de 2022

ALTO

Front

São 18.936 linhas de TypeScript sobre uma base parada: **Next 12.2.2** (julho de 2022, sem suporte), `axios@0.27.2` e Node 18 no CI, também em fim de vida. A biblioteca `material-ui@0.20.2`, anterior ao MUI, convive com `@mui/material@5` — peso morto no bundle. O `tsconfig.json` ainda compila para `es5`.

CRITÉRIO DE ACEITE

Next em versão suportada, `material-ui` legado removido, alvo de compilação atualizado e suíte verde.

Esforço 2 semanas package.json · tsconfig.json

MSG-34 Um terço do código de testes do Front é snapshot

ALTO

Front

São **8.887 linhas em 37 arquivos** `.snap`, quase um terço das 27.798 linhas de `src/`. Dos 290 `expect()` da suíte, **61 são** `toMatchSnapshot()`.

Teste de snapshot detecta que algo mudou, não que algo *quebrou*. Na prática, quem mexe no componente atualiza o snapshot sem ler o diff — **a cobertura reportada é maior do que a proteção real**. É o que faz o número de cobertura do Front não ser comparável ao do Core.

CRITÉRIO DE ACEITE

Snapshots substituídos por asserções de comportamento nos componentes de maior risco, começando pelo fluxo de criação de release.

Esforço **2 semanas** `src/**/*.tests/__snapshots__/`

MSG-35 96 usos de `any` e 58 supressões de lint no Front

MÉDIO

Front

O `tsconfig.json` tem `strict: true`, mas o efeito é anulado por **96 ocorrências de** `any` e **58 de** `@ts-ignore` ou `eslint-disable`. Cada uma é um ponto onde o compilador deixou de proteger.

CRITÉRIO DE ACEITE

Regra de lint que impede novas ocorrências, e redução progressiva das existentes começando pela camada de serviços.

Esforço **1 semana** `src/` (Front)

MSG-36 Suíte do Service exige Postgres e não roda localmente sem Docker

MÉDIO

Service

São **274 testes definidos** que não puderam ser executados neste diagnóstico por dependerem de um Postgres em pé. O `Makefile` cobre bem o caso com Docker, mas não há caminho alternativo — **a barreira de entrada para contribuir sobe**, e a cobertura real do maior repositório do projeto fica não verificada fora do CI.

CRITÉRIO DE ACEITE

Suíte executável sem Docker, ou documentação explícita de que o Docker é pré-requisito com diagnóstico claro quando ausente.

Esforço **1 dia** `tox.ini` · `Makefile`

MSG-37 `pytest.ini` do Core se contradiz

BAIXO

Core

```
addopts = -ra -q --ignore=./tests    ← ignora o diretório tests
testpaths =
    tests                             ← ...e manda procurar em tests
```

Só funciona porque o tox invoca `pytest tests` explicitamente. Quem roda `pytest` puro esbarra em comportamento confuso — foi preciso `PYTHONPATH=src` manual para executar a suíte neste diagnóstico. Atrito de integração para quem chega.

CRITÉRIO DE ACEITE

`pytest` sem argumentos roda a suíte inteira a partir da raiz do repositório.

Esforço **1 h** `pytest.ini` · `tox.ini`

MSG-38 Notebook de 63 KB versionado na raiz do Core

BAIXO

Core

`entendendo_as Estruturas de dados.ipynb` tem valor didático, mas notebooks versionados carregam saídas e metadados de execução que produzem diffs ilegíveis. O conteúdo pertence ao repositório de documentação.

CRITÉRIO DE ACEITE

Conteúdo migrado para os Docs, ou notebook mantido com saídas limpas por *hook* de pré-commit.

Esforço **2 h** `entendendo_as Estruturas de dados.ipynb`

E8 Governança e identidade do projeto

Itens que não são código, mas que produzem código ruim. É aqui que está a causa de boa parte do backlog acima.

MSG-39 Duas identidades concorrentes: repositórios centrais versus de semestre

ALTO

Transversal

A documentação de julho de 2026 lista **nove repositórios ativos com prefixo 2026.1-**, incluindo **AI** e **Plugin**, que não existem no conjunto central. As próprias páginas dos Docs referenciam *ambos* os padrões de nome, e ainda um `MeasureSoftGram-Helper-Action` que não aparece em lugar nenhum.

É a causa-raiz de MSG-22 (chaves do Sonar divergentes), de MSG-04 (código publicado fora do repositório central) e da confusão de qualquer pessoa que chegue ao projeto.

CRITÉRIO DE ACEITE

Decisão registrada em ADR sobre qual conjunto é canônico, repositórios não canônicos arquivados e documentação alinhada.

Esforço 1 semana Docs · organização no GitHub

MSG-40 O Front em produção não corresponde ao código versionado

ALTO

Front

O workflow de deploy do Service e a configuração nginx de produção roteiam um contêiner `front:3000` e registram que “o Front tem o seu proprio deploy + runner”. Mas o repositório central do Front **não recebe commit real desde setembro de 2024**.

Existe um frontend em produção cuja origem não está identificada. Qualquer correção feita no repositório central hoje **não chega ao ar**.

CRITÉRIO DE ACEITE

Origem do artefato em produção identificada e reconciliada com o repositório canônico, com pipeline de deploy documentado.

Esforço 3 dias deploy/ · MeasureSoftGram-Front

MSG-41 **Squash de fim de semestre apaga a autoria do trabalho**

MÉDIO

Transversal

Os commits `Sincronização de repositório - 2026.1` trazem 2.065 linhas em 45 arquivos (CLI) e centenas de arquivos (Service) **em um único commit**. Autoria individual, motivação de cada mudança e possibilidade de bissecção desaparecem.

Some-se a isso que os branches divergiram: `main` tem **13 commits que não estão em** `develop` no CLI, 9 no Service e 7 no Core, enquanto o `develop` do Front está 1.000 commits à frente do `main`. **Não há um branch que represente o estado do projeto.**

CRITÉRIO DE ACEITE

Política de merge registrada, branches reconciliados e sincronização de semestre feita por PRs temáticos com histórico preservado.

Esforço **1 semana** política de branches · 6 repos